

Training Neural Networks on a GPU with PyTorch

Device-Safe Data Transfer, Efficient Mini-Batches, and Correct Evaluation

Abstract

A graphics processing unit (GPU) can execute the many independent tensor operations in neural-network training with high parallel throughput. Correct acceleration requires more than selecting a GPU runtime: PyTorch must expose the accelerator, the model and every participating tensor must occupy the same device, host-to-device transfers must occur at deliberate boundaries, and timing must account for asynchronous execution. These notes scale a Fashion-MNIST multilayer perceptron from a 6,000-row CPU baseline to a 60,000-row table, derive every important shape and batch count, implement device-agnostic training and evaluation loops, explain pinned host memory and batch-size trade-offs, and distinguish a speed improvement from an accuracy improvement.

Contents

1	Purpose and complete mental map	2
2	Data scale, split arithmetic, and experiment meaning	3
2.1	From a subset to the full training table	3
2.2	Load, split, and scale the table	3
2.3	Mini-batch arithmetic	4
3	Selecting an accelerator without losing portability	5
3.1	Runtime setup and availability	5
4	The same-device invariant	6
4.1	Check the invariant directly	6
5	Datasets, loaders, and the model's final placement	7
5.1	CPU-side sample access	7
5.2	Move the model once	7
6	Device-aware mini-batch training	9
6.1	State transition for one batch	9
6.2	Runnable training loop	10
7	Evaluation must obey the same placement rule	11
7.1	Crossing back to a CPU consumer	11
8	Pinned host memory and non-blocking transfer	12
8.1	Pageable versus pinned memory	12

9	Batch size is a throughput–memory–optimization trade-off	13
10	Measure accelerator time correctly	14
11	Interpreting the reported training result	15
11.1	Observed optimization trace	15
12	Common failures and diagnostic repairs	17
13	Key takeaways	18
14	Review questions	18

1 Purpose and complete mental map

The practical purpose is to reduce wall-clock time when training on a much larger collection or repeating many experiments. Four terms establish the mental model.

CPU A *central processing unit*; a general-purpose processor optimized for low-latency execution of varied instructions and control flow.

GPU A *graphics processing unit*; a processor containing many execution units that can apply similar numerical operations to many data elements in parallel.

CUDA NVIDIA’s accelerator programming platform. In PyTorch, a CUDA device is addressed with a name such as `cuda` or `cuda:0`.

Device The memory and compute location attached to a tensor or module, most commonly `cpu` or `cuda:0`. A device is part of a tensor’s state alongside its shape and data type.

The CPU still performs useful work: reading data, sampling indices, constructing mini-batches, and coordinating the program. Each mini-batch is copied to accelerator memory, where the model’s forward pass, loss calculation, backward pass, and parameter update occur. Small scalar metrics may then return to Python on the CPU.

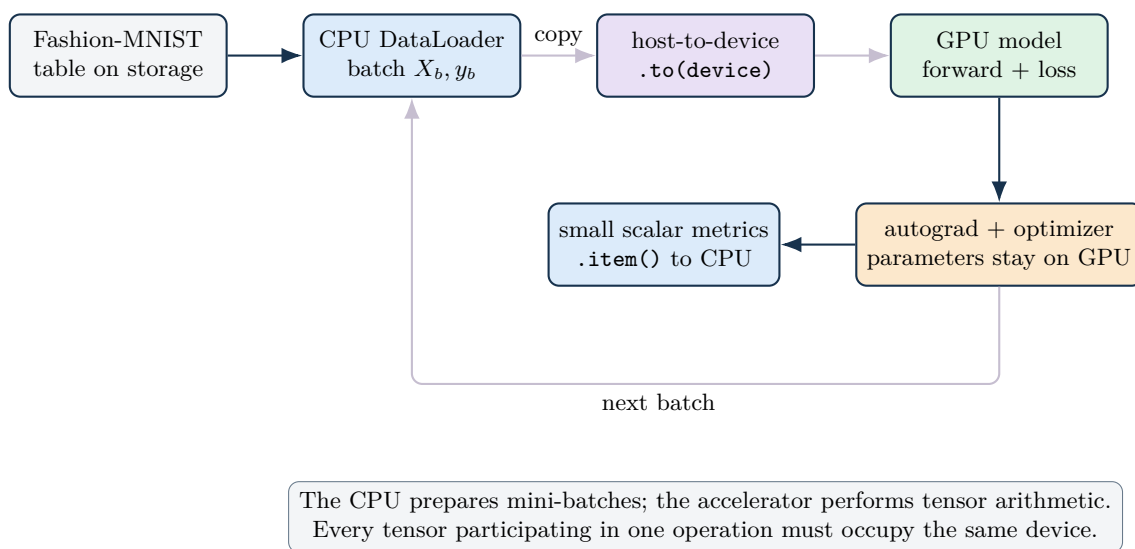


Figure 1: A device-aware pipeline separates CPU data preparation, host-to-device transfer, accelerator computation, and small metric transfers. [Technical reference](#).

Intuition

A GPU does not make Python syntax different and does not automatically improve a model's statistical quality. It changes where tensor operations execute and can reduce the time required to perform the same mathematical workload.

Takeaway

Acceleration is a data-placement problem as well as a compute problem. The model, feature batch, and label batch must be moved deliberately, while the DataLoader ordinarily remains a CPU-side producer.

2 Data scale, split arithmetic, and experiment meaning

2.1 From a subset to the full training table

Fashion-MNIST contains 70,000 grayscale images in its canonical distribution: 60,000 training images and a separate 10,000-image test set. Each image has $28 \times 28 = 784$ pixel values and one class label. The expanded workflow uses a 60,000-row table and then applies a new 80:20 split to that table:

$$N_{\text{train}} = 0.8 \times 60,000 = 48,000, \quad N_{\text{held}} = 0.2 \times 60,000 = 12,000.$$

Here N_{train} is the number of rows used for parameter updates and N_{held} is the number used for held-out evaluation. The separate canonical 10,000-image test file does not enter this particular re-split.

The preceding small experiment used 6,000 rows, producing 4,800 training rows and 1,200 held-out rows. The scale change is therefore tenfold in both partitions:

$$\frac{48,000}{4,800} = 10, \quad \frac{12,000}{1,200} = 10.$$

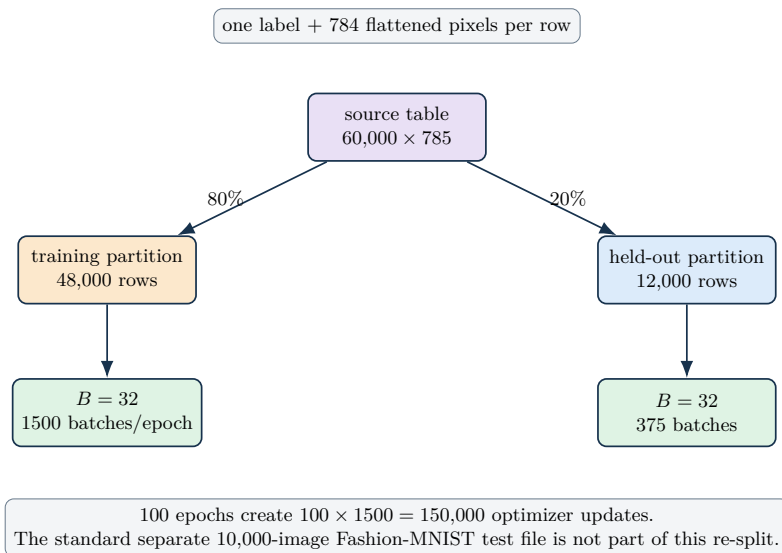


Figure 2: An 80:20 split of the 60,000-row table creates 48,000 training rows, 12,000 held-out rows, and exact batch counts for $B = 32$. [Technical reference](#).

2.2 Load, split, and scale the table

A table row has one label column and 784 feature columns. The resulting table shape is $[60,000, 785]$. The demonstrated CSV upload was approximately 126.88 MB, which is large enough to make data-loading

and transfer choices visible in practice. Pixel scaling maps integer intensities from 0 through 255 to floating-point values in $[0, 1]$.

```
import pandas as pd
from sklearn.model_selection import train_test_split

frame = pd.read_csv("fashion-mnist_train.csv")
X = frame.iloc[:, 1:].to_numpy()
y = frame.iloc[:, 0].to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
)
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

assert X_train.shape == (48_000, 784)
assert X_test.shape == (12_000, 784)
```

Listing 1: Load the full Fashion-MNIST training table and create an 80:20 split

The split code is reproducible because `random_state=42` fixes the scikit-learn pseudorandom sequence. Before splitting, a useful sanity check reshapes each of the first sixteen 784-entry feature rows with `reshape(28, 28)` and places them in a `plt.subplots(4, 4)` grid. This confirms that flattening order, pixel range, and label alignment are plausible before the larger run begins. A stronger classification protocol can add `stratify=y` so both partitions preserve class proportions. If hyperparameters will be changed repeatedly, the 12,000-row partition should be treated as validation data and a separate untouched test set should be retained for the final estimate.

2.3 Mini-batch arithmetic

For batch size $B = 32$, both partition sizes are exact multiples of B :

$$\frac{48,000}{32} = 1500 \quad \text{training batches per epoch}, \quad \frac{12,000}{32} = 375 \quad \text{evaluation batches.}$$

With $E = 100$ epochs, where an *epoch* is one pass over all training rows, ordinary stochastic gradient descent performs

$$E \times 1500 = 100 \times 1500 = 150,000$$

optimizer updates. The symbol E denotes epoch count and each update follows one mini-batch backward pass.

One feature batch in 32-bit floating-point format occupies

$$32 \times 784 \times 4 = 100,352 \text{ bytes.}$$

A 32-label batch in 64-bit integer format occupies

$$32 \times 8 = 256 \text{ bytes.}$$

The direct feature-plus-label payload is therefore 100,608 bytes, or 98.25 KiB. This excludes Python objects, allocator bookkeeping, model activations, gradients, and temporary workspaces.

Takeaway

The expanded run changes both the amount of training data and the execution device. More data can affect generalization; the GPU primarily makes the larger computation practical.

3 Selecting an accelerator without losing portability

3.1 Runtime setup and availability

A hosted notebook must first be configured with a GPU accelerator through its runtime settings. That user-interface selection only requests an accelerator. The program must still verify that the current PyTorch process can use CUDA.

```
import torch

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("selected device:", device)
print("PyTorch build CUDA version:", torch.version.cuda)
if device.type == "cuda":
    print("accelerator:", torch.cuda.get_device_name(0))
```

Listing 2: Select CUDA when available and retain a CPU fallback

`torch.cuda.is_available()` returns a Boolean value. A result of `False` can mean that no supported NVIDIA GPU exists, the driver is unavailable, the environment hides the device, or a CPU-only PyTorch build is installed. A device-agnostic program remains correct by selecting `cpu` in those cases.

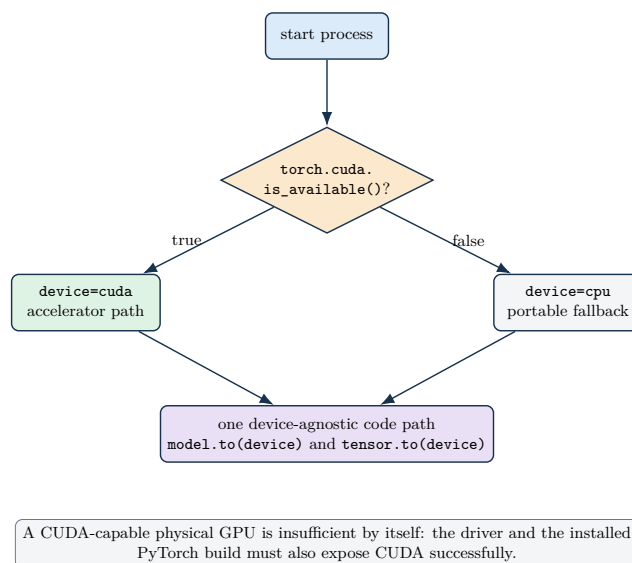


Figure 3: The availability check selects one device object while preserving a single code path for CUDA and CPU execution. [Technical reference](#).

Pitfall

Do not hard-code `torch.device("cuda")` in code that must run everywhere. A hard-coded CUDA device causes startup failure on a CPU-only host, while a checked fallback permits correctness testing even when accelerator performance cannot be measured.

4 The same-device invariant

Suppose a mini-batch contains B examples. Let

$$X_b \in \mathbb{R}^{B \times 784}, \quad y_b \in \{0, \dots, 9\}^B,$$

where X_b is the feature matrix and y_b is the class-index vector. Let θ denote every trainable model parameter and let d be the selected device. Valid execution requires

$$\text{device}(\theta) = \text{device}(X_b) = \text{device}(y_b) = d.$$

This equality matters because the forward pass combines X_b with weights in θ , and cross-entropy combines model logits with y_b .

For a two-example manual shape trace,

$$X_b : [2, 784] \xrightarrow{\text{model on } d} Z_b : [2, 10], \quad y_b : [2].$$

The logit matrix Z_b and label vector y_b must share d before the scalar loss $\mathcal{L}(Z_b, y_b)$ can be computed.

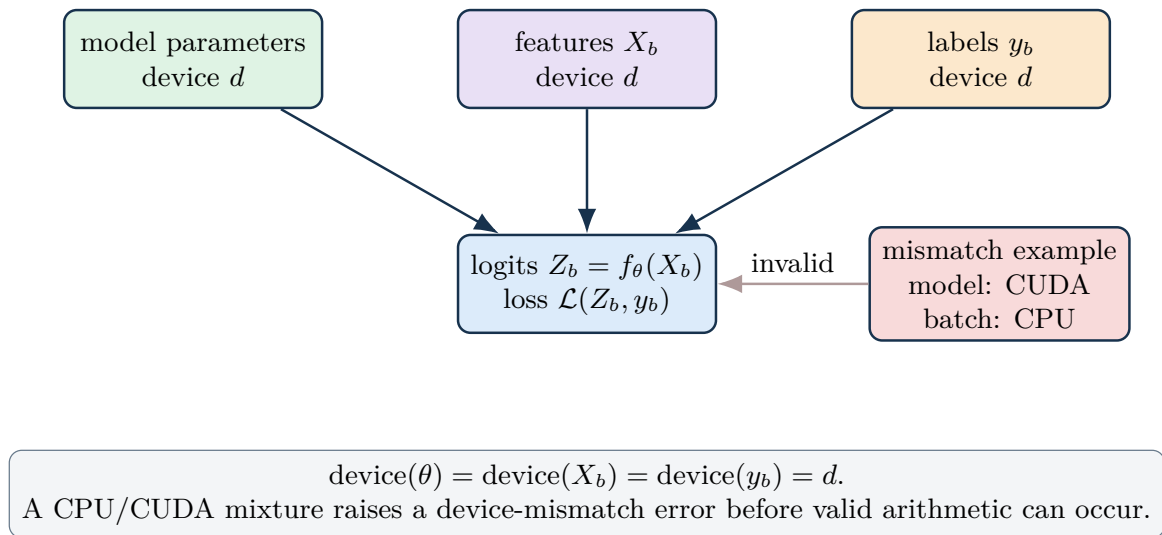


Figure 4: Model parameters, input features, and target labels must share one device before forward or loss arithmetic occurs. [Technical reference](#).

4.1 Check the invariant directly

```
def check_batch(model, features, labels, device):
    parameter_device = next(model.parameters()).device
    assert parameter_device == device
    assert features.device == device
    assert labels.device == device
    assert features.ndim == 2
    assert features.shape[1] == 784
    assert features.dtype == torch.float32
    assert labels.ndim == 1
    assert labels.dtype == torch.long
    assert labels.shape[0] == features.shape[0]
```

Listing 3: Verify shape, dtype, and device contracts before a long run

A mismatch typically raises an error containing wording such as “expected all tensors to be on the same device.” That failure is useful: silently mixing memory domains would not define a valid matrix multiplication.

5 Datasets, loaders, and the model's final placement

5.1 CPU-side sample access

A map-style Dataset owns aligned feature and label tensors. A DataLoader requests samples, collates them into batches, and returns CPU tensors unless a specialized custom path does otherwise.

```
from torch.utils.data import Dataset, DataLoader

class CustomDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(
            features, dtype=torch.float32
        )
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        return self.features[index], self.labels[index]

train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)
train_loader = DataLoader(
    train_dataset, batch_size=32, shuffle=True
)
test_loader = DataLoader(
    test_dataset, batch_size=32, shuffle=False
)
```

Listing 4: Create tensor-backed datasets and CPU-side mini-batch loaders

The ordinary batch contracts are

`batch_features : [32, 784] float32,      batch_labels : [32] int64.`

Feature values occupy one row per example. Label values occupy one scalar per example; a `[32, 1]` target would be the wrong shape for class-index cross-entropy.

5.2 Move the model once

The classifier has shape flow

$$[B, 784] \rightarrow [B, 128] \rightarrow [B, 128] \rightarrow [B, 64] \rightarrow [B, 64] \rightarrow [B, 10].$$

The repeated shapes arise because each ReLU changes values but preserves dimensions.

```

import torch.nn as nn
import torch.optim as optim

class MyNN(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(num_features, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10),
        )

    def forward(self, features):
        return self.model(features)

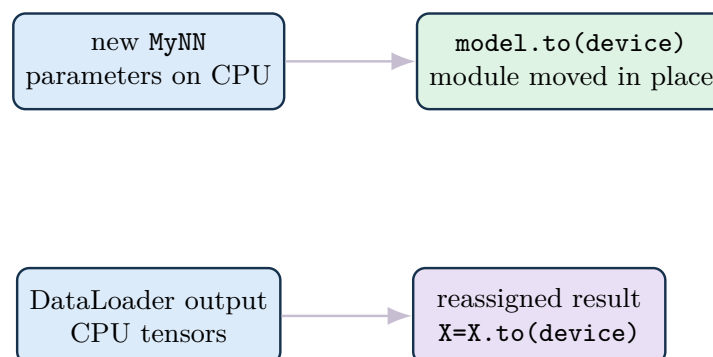
model = MyNN(num_features=784).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)

```

Listing 5: Define the classifier, place it on the final device, and then construct the optimizer

`nn.Module.to(device)` moves registered parameters and buffers and returns the module itself. Calling it before optimizer construction is the safest general ordering because the optimizer then receives parameters in their final placement. The model is moved once, outside all batch loops.

A tensor's `.to(device)` method has different assignment semantics: it returns a tensor on the requested device. If the original tensor is not already suitable, failing to retain that return value leaves the named variable on its old device.



A tensor transfer returns a tensor, so retain the result. A module transfer updates registered parameters and buffers; construct the optimizer after placing the module on its final device.

Figure 5: Module placement and mini-batch placement use similar syntax but have different lifecycle and assignment responsibilities. [Technical reference](#).

The model contains 109,386 trainable scalar parameters. In 32-bit floating-point storage, parameters alone require

$$109,386 \times 4 = 437,544 \text{ bytes} \approx 427.29 \text{ KiB.}$$

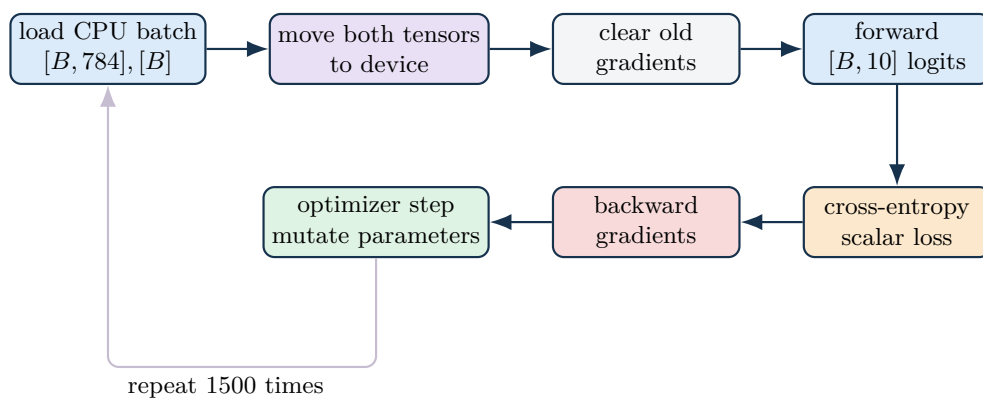
Training memory is larger because gradients, activations, temporary buffers, and optional optimizer state must also be stored.

6 Device-aware mini-batch training

6.1 State transition for one batch

For one mini-batch, the algorithm has six ordered responsibilities:

1. copy features and labels from host memory to the selected device;
2. clear gradients left by the preceding backward pass;
3. compute logits with a forward pass;
4. compute scalar mean cross-entropy loss;
5. run backpropagation to populate parameter gradients;
6. mutate parameters with the optimizer.



Model parameters remain on the accelerator across iterations. Only new mini-batches cross the host–device boundary; moving the model inside the loop would waste work.

Figure 6: Each training iteration transfers a fresh batch, computes and differentiates loss on one device, and updates persistent model parameters. [Technical reference](#).

6.2 Runnable training loop

```

epochs = 100
model.train()

for epoch in range(epochs):
    loss_sum = 0.0
    example_count = 0

    for batch_features, batch_labels in train_loader:
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        optimizer.zero_grad(set_to_none=True)
        logits = model(batch_features)
        loss = criterion(logits, batch_labels)
        loss.backward()
        optimizer.step()

        batch_count = batch_labels.shape[0]
        loss_sum += loss.item() * batch_count
        example_count += batch_count

    mean_loss = loss_sum / example_count
    print(f"epoch={epoch + 1:03d} loss={mean_loss:.4f}")

```

Listing 6: Train for 100 epochs with model and mini-batches on the selected device

The forward equation for the first affine layer is

$$H_1 = \text{ReLU}(X_b W_1^T + b_1).$$

Here $X_b \in \mathbb{R}^{B \times 784}$, $W_1 \in \mathbb{R}^{128 \times 784}$, $b_1 \in \mathbb{R}^{128}$, and $H_1 \in \mathbb{R}^{B \times 128}$. The bias vector is broadcast across all B rows. Every operand resides on the selected device.

The mean-loss calculation multiplies each batch mean by its actual sample count before summing. This remains correct even when a dataset produces a smaller final batch. For the current 48,000-row training partition and $B = 32$, every batch is complete, so the weighted and unweighted means coincide.

One observed 100-epoch run began with the following mean mini-batch losses:

Epoch	Reported loss
1	0.6352872455
2	0.4304986934
3	0.3861262033
4	0.3584607187
5	0.3376494686
6	0.3227646786

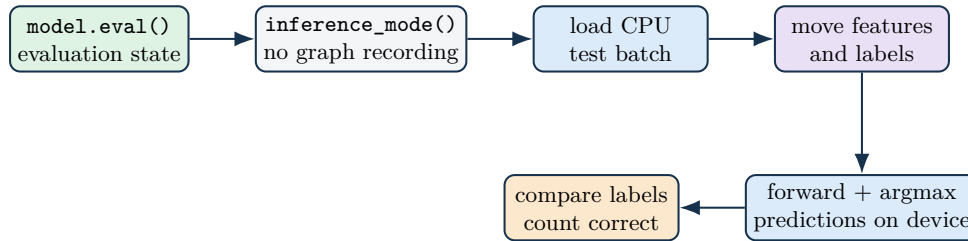
These values document one stochastic optimization path rather than universal expected output. The strong initial decrease confirms that the full-data device-aware loop was updating parameters in a useful direction; later fluctuations remain possible.

Pitfall

Moving only `batch_features` is insufficient. Cross-entropy combines accelerator logits with `batch_labels`; labels left on the CPU cause a device-mismatch failure during loss computation.

7 Evaluation must obey the same placement rule

`model.eval()` changes the behavior of stateful layers such as dropout and batch normalization. It does not disable autograd. `torch.inference_mode()` separately prevents graph recording and additional autograd bookkeeping. They solve different problems and should be used together for ordinary inference.



Transfer labels as well as features: prediction-label comparison is tensor arithmetic. Move outputs to CPU only when a CPU consumer such as NumPy actually needs them.

Figure 7: Evaluation selects inference behavior, suppresses gradient recording, transfers both tensors, and counts predictions on the selected device. [Technical reference](#).

```

model.eval()
correct = 0
total = 0

with torch.inference_mode():
    for batch_features, batch_labels in test_loader:
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        logits = model(batch_features)
        predicted = logits.argmax(dim=1)

        correct += (predicted == batch_labels).sum().item()
        total += batch_labels.numel()

accuracy = correct / total
print(f"accuracy={accuracy:.4f}")

```

Listing 7: Evaluate sample-level accuracy with features, labels, and predictions on one device

For N evaluated examples, accuracy is

$$\text{accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i].$$

The symbol $\hat{y}_i$ is the predicted class, y_i is the true class, and $\mathbf{1}[\cdot]$ equals 1 when its condition is true and 0 otherwise. If 9 of 10 predictions are correct, then accuracy is $9/10 = 0.9 = 90\%$.

`argmax(dim=1)` searches across the ten class columns of a $[B, 10]$ logit matrix and returns a $[B]$ class-index vector. Softmax is unnecessary for class selection because it preserves the ordering of logits.

7.1 Crossing back to a CPU consumer

Python scalars can be obtained with `.item()`. NumPy arrays require CPU memory, so accelerator tensors must be detached and moved first.

```
probabilities = torch.softmax(logits, dim=1)
probabilities_numpy = probabilities.detach().cpu().numpy()
```

Listing 8: Convert inference results safely for a CPU-only NumPy consumer

Calling `.numpy()` directly on a CUDA tensor is invalid because NumPy does not operate on CUDA memory. Calling it on a tensor that requires gradients is also invalid unless the tensor is detached.

8 Pinned host memory and non-blocking transfer

8.1 Pageable versus pinned memory

Ordinary CPU tensor storage is usually *pageable*: the operating system may relocate or swap its physical pages. Direct memory access hardware needs stable source addresses while copying to the GPU. A pageable transfer can therefore require an intermediate copy into a page-locked, or *pinned*, staging region. A pinned page cannot be swapped out during the transfer. DataLoader can pin each returned batch by setting `pin_memory=True`. When the source batch is pinned, `tensor.to(device, non_blocking=True)` can request an asynchronous host-to-device copy.

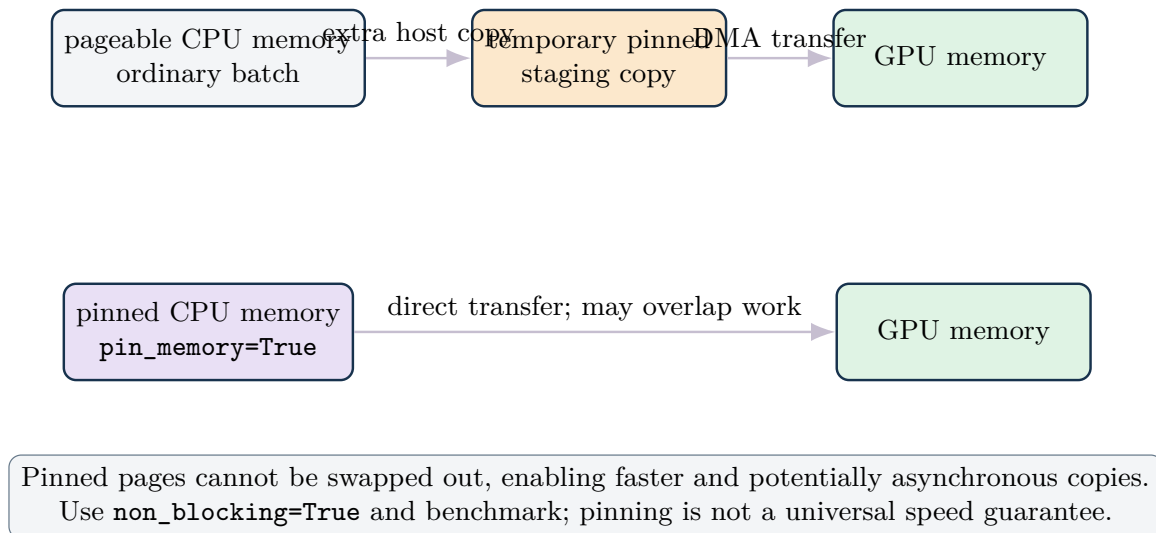


Figure 8: Pinned host batches remove an extra staging step and can support asynchronous transfer to accelerator memory. [Technical reference](#).

```

use_cuda = device.type == "cuda"

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    pin_memory=use_cuda,
)

def move_batch(features, labels):
    features = features.to(
        device, non_blocking=use_cuda
    )
    labels = labels.to(
        device, non_blocking=use_cuda
    )
    return features, labels

```

Listing 9: Enable pinned batches and request non-blocking CUDA transfers

The Boolean `use_cuda` avoids requesting pinning on a CPU-only run. The helper returns both moved tensors so callers cannot accidentally discard the result of `.to()`.

Pitfall

Pinned memory is a limited operating-system resource. Pinning an entire large dataset manually can reduce system responsiveness. `Dataloader`'s batch-level pinning is usually preferable. Its benefit also depends on transfer size, worker behavior, model compute, and hardware; benchmark the complete pipeline rather than assuming a fixed speedup.

Asynchronous copies do not automatically overlap useful work. Effective overlap depends on pinned source memory, available copy engines, stream scheduling, and enough accelerator computation to hide transfer latency. For small batches or a small model, launch and transfer overhead can dominate.

9 Batch size is a throughput–memory–optimization trade-off

A larger batch can expose more parallel work to the accelerator and reduce the number of optimizer updates per epoch. It also consumes more memory and changes the statistical noise of the gradient estimate.

The loader configuration can be changed as a controlled experiment. The following version uses 128 examples per complete batch while retaining portable conditional pinning.

```

train_loader = DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True,
    pin_memory=use_cuda,
)
test_loader = DataLoader(
    test_dataset,
    batch_size=128,
    shuffle=False,
    pin_memory=use_cuda,
)

```

Listing 10: Construct 128-example loaders for a measured accelerator-utilization experiment

For 48,000 training rows and 12,000 held-out rows:

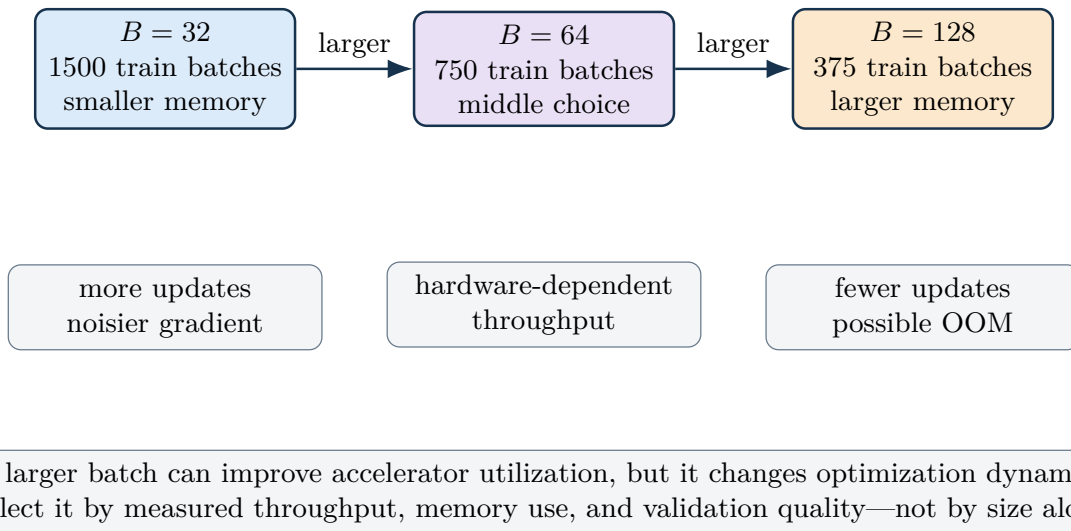


Figure 9: Batch sizes 32, 64, and 128 trade update count, transfer payload, memory use, and hardware utilization. [Technical reference](#).

B	Train batches	Held-out batches	Feature payload per complete batch
32	1500	375	$32 \cdot 784 \cdot 4 = 100,352$ bytes
64	750	188	$64 \cdot 784 \cdot 4 = 200,704$ bytes
128	375	94	$128 \cdot 784 \cdot 4 = 401,408$ bytes

The held-out counts for $B = 64$ and $B = 128$ use ceiling division because 12,000 is not divisible by either size. For example,

$$12,000 = 93 \times 128 + 96,$$

so $B = 128$ produces 93 complete batches and one final 96-example batch, for 94 batches total.

Batch size 64 or 128 may outperform 32 on a particular GPU, but no value is universally best. A disciplined comparison records examples per second, peak device memory, training loss, validation quality, and random seed. If an out-of-memory error occurs, reduce batch size, simplify the model, use mixed precision when justified, or deliberately accumulate gradients across smaller batches.

10 Measure accelerator time correctly

CUDA kernel launches are normally asynchronous with respect to the CPU. A CPU wall-clock timer can stop before queued GPU work has completed. Synchronize at measurement boundaries or use CUDA events.

```

import time

def elapsed_seconds(fn, device):
    if device.type == "cuda":
        torch.cuda.synchronize()
        start = torch.cuda.Event(enable_timing=True)
        end = torch.cuda.Event(enable_timing=True)
        start.record()
        fn()
        end.record()
        torch.cuda.synchronize()
        return start.elapsed_time(end) / 1000.0

    start = time.perf_counter()
    fn()
    return time.perf_counter() - start

```

Listing 11: Measure one callable with synchronization-aware device timing

A fair benchmark also includes warm-up iterations, repeats measurements, reports a distribution rather than one value, and states whether data loading and host-to-device transfer are inside the timed region. GPU acceleration may be slower for tiny workloads because kernel-launch and transfer overhead outweigh parallel computation.

11 Interpreting the reported training result

The expanded configuration completed 100 epochs on 48,000 training rows. Its final displayed mean mini-batch loss was 0.06209626051294617, and held-out accuracy was

$$0.8898333333333334 = \frac{10,678}{12,000} \approx 88.9833\%.$$

The preceding 6,000-row configuration reported accuracy 0.8325. The absolute change is

$$0.8898333333333334 - 0.8325 = 0.0573333333333335,$$

or approximately 5.7333 *percentage points*. A percentage-point change is an absolute difference between rates; it is not the same as the relative increase, which is approximately 6.8869% of the baseline rate.

Two conclusions are justified:

1. GPU execution made the much larger training workload fast enough to run conveniently.
2. Using ten times as many rows can supply more variation from which the model may learn.

One conclusion is *not* justified: the accuracy gain cannot be attributed to the GPU itself because data quantity changed at the same time. A controlled device comparison fixes the data, split, initialization seed, model, batch order, optimizer, learning rate, epoch count, and numerical precision, then changes only CPU versus GPU. Small numerical differences can still occur because hardware backends may use different algorithms and floating-point reduction orders.

Further improvement can investigate weight initialization, regularization, optimizer choice, learning-rate schedules, architecture, and systematic hyperparameter tuning. A natural next milestone is to exceed 90% held-out accuracy under a fixed validation protocol. These choices should be selected on validation data rather than by repeatedly inspecting the final test set.

11.1 Observed optimization trace

The following table preserves every epoch/loss line that was legible in the demonstrated output. Epoch 7 and epochs 57–83 were not legible in the available views and are intentionally omitted rather than

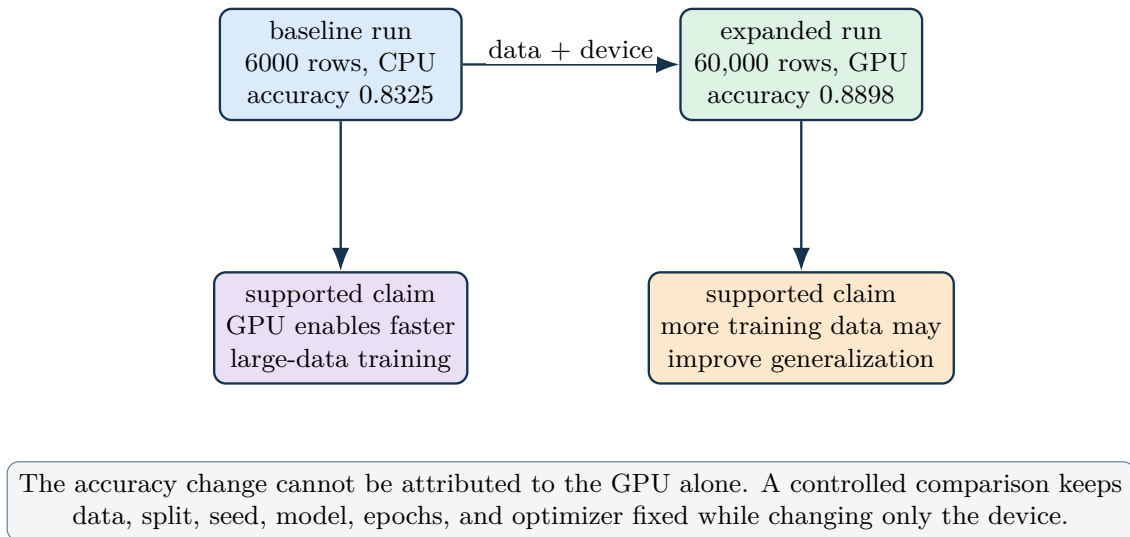


Figure 10: The expanded run changes both data scale and compute device, so speed and accuracy claims must be separated. [Technical reference](#).

reconstructed. The trace demonstrates a strong long-term decline together with local increases, such as epochs 84–85 and 99–100.

Epoch	Loss	Epoch	Loss	Epoch	Loss
1	0.6352872455318769	2	0.4304986933817466	3	0.38612620331346986
4	0.3584607186516126	5	0.3376494685560465	6	0.3227646785800656
8	0.29498188248028356	9	0.2854692431092262	10	0.27467057487616936
11	0.26830568559716145	12	0.25814215187107525	13	0.24940819021190205
14	0.24444738084822892	15	0.23859191479658087	16	0.23155898495018482
17	0.22562562109405795	18	0.220296459933122	19	0.21206333881616593
20	0.20960091097543953	21	0.20624992236122489	22	0.19986102367161462
23	0.19530413204431535	24	0.19312162966746838	25	0.18764106344679993
26	0.1836644180783381	27	0.1801829884586235	28	0.1732649680164953
29	0.17048048644885422	30	0.1704416691598793	31	0.1669696749355644
32	0.16373400872449081	33	0.16233399505944301	34	0.15723733726826808
35	0.15447346428254	36	0.1545576546518132	37	0.14620412410050632
38	0.14241028870362787	39	0.14137836676385876	40	0.1423039033394307
41	0.1375529004293494	42	0.13899335852917283	43	0.13538831498442838
44	0.1287984882996728	45	0.12960014000659187	46	0.12750795842163887
47	0.12409021653390179	48	0.12071532321178044	49	0.11564283333004763
50	0.11969620903915104	51	0.11240664103456463	52	0.11313746599317528
53	0.11436900947755203	54	0.11118536476511508	55	0.1151556797324059
56	0.10828998744379108	57	0.106879183093741692	58	0.108118748380765707
59	0.107770356753194937	60	0.106794948129361728	61	0.106668691740721502
62	0.106795124354235305	63	0.107117577052312844	64	0.106771438679029719
65	0.106959606989556535	66	0.106168472910565712	67	0.106411504278382442
68	0.106060305817479578	69	0.105729881219780267	70	0.1058360233872032645
71	0.106350834116928429	72	0.10521324163050546	73	0.106209626051294617

Takeaway

Compute speed and predictive quality are different outcome variables. Device choice changes execution performance; data, optimization, regularization, and architecture govern the statistical experiment.

12 Common failures and diagnostic repairs

1. **CUDA is unavailable.** Inspect `torch.cuda.is_available()`, `torch.version.cuda`, the driver, hosted runtime selection, and installed PyTorch build. Retain a CPU fallback.
2. **Model remains on CPU.** Call `model.to(device)` once before constructing the optimizer and before the first forward pass.
3. **Tensor transfer result is discarded.** Assign `x = x.to(device)` or return the moved tensor from a helper.
4. **Labels remain on CPU.** Move both features and class-index labels before cross-entropy and prediction comparison.
5. **A hidden tensor is created on the wrong device.** Create masks, constants, and recurrent states with an explicit device or derive them with methods such as `x.new_zeros(...)`.
6. **Wrong feature data type.** Linear layers usually expect floating-point features. Convert image features to `float32`.
7. **Wrong label data type or shape.** Multiclass class-index targets for cross-entropy use `long` and shape `[B]`.
8. **Direct NumPy conversion fails.** Use `tensor.detach().cpu().numpy()` for a CPU NumPy consumer.
9. **GPU memory is exhausted.** Reduce batch size, inspect retained computation graphs, delete obsolete references, or use an appropriate memory-saving method. Do not call `empty_cache()` as a substitute for fixing live allocations.
10. **Pinned memory is enabled without benefit.** Measure the end-to-end pipeline and avoid excessive pinned storage or worker counts.
11. **Timing reports implausibly tiny values.** Synchronize CUDA or use CUDA events at the measurement boundaries.
12. **GPU utilization remains low.** Profile data loading, preprocessing, transfer, batch size, and model size; the accelerator may be waiting for the host.
13. **The test partition guides every tuning decision.** Introduce a validation partition and reserve an untouched test set for final reporting.

```
features, labels = next(iter(train_loader))
features = features.to(device)
labels = labels.to(device)

check_batch(model, features, labels, device)
logits = model(features)

assert logits.shape == (features.shape[0], 10)
assert logits.device == device
assert sum(p.numel() for p in model.parameters()) == 109_386
assert all(p.device == device for p in model.parameters())
```

Listing 12: Assert the critical contracts immediately before training

13 Key takeaways

1. GPU training accelerates compatible tensor computation; it does not automatically improve accuracy.
2. The 60,000-row table becomes 48,000 training rows and 12,000 held-out rows under an 80:20 split.
3. Batch size 32 creates 1,500 training batches and 375 held-out batches.
4. The model, feature batch, and label batch must occupy the same selected device.
5. A module is moved once; fresh mini-batches are moved inside their loops.
6. Tensor `.to()` calls must retain the returned tensor.
7. Evaluation state and disabled gradient recording are complementary controls.
8. Pinned host memory can improve transfer behavior, but its benefit is workload- and hardware-dependent.
9. Larger batches trade memory and optimization behavior for potential throughput.
10. CUDA timing requires synchronization or CUDA events.
11. A comparison that changes both data quantity and device cannot isolate either factor's effect on accuracy.

14 Review questions

1. What does a PyTorch tensor's device represent, and why is it part of the tensor's operational state?
2. Why can a hosted notebook display a GPU option while `torch.cuda.is_available()` still returns `False`?
3. Derive the 48,000/12,000 split and the 1,500/375 batch counts.
4. What shapes and data types should a 32-example feature batch and label batch have?
5. Why must labels be transferred even though they contain only integer class identifiers?
6. How does moving a tensor differ from moving a module? Why must a tensor transfer result be retained?
7. Why should the model normally be moved before optimizer construction?
8. Which operations remain on the GPU during one training iteration?
9. Why are `model.eval()` and `torch.inference_mode()` not interchangeable?
10. What extra step is required before converting a CUDA tensor to NumPy?
11. How can pinned memory reduce transfer overhead, and why can too much pinning be harmful?
12. Why might batch size 128 be faster, slower, or less accurate than batch size 32?
13. Why can an unsynchronized CPU timer understate CUDA execution time?
14. Which controlled experiment would separate the effect of more data from the effect of GPU execution?
15. Why is an increase from 83.25% to approximately 88.98% described as about 5.73 percentage points rather than a 5.73% relative increase?